

Build a Python To-Do App

and Push it to GitHub — Complete Step-by-Step Guide

Zero experience needed Every line explained Done in 3-4 hours

WHAT YOU WILL BUILD TODAY

A working command-line To-Do app that saves tasks to a file,
visible on GitHub with a live link you can add to your resume.

5 Steps: Install Python -> Write App -> Test -> GitHub -> Upload

This guide assumes ZERO prior experience. Every step is explained in plain English. Every line of code has a plain-English explanation. Follow page by page and you will have a working project on GitHub by tonight.

How long each step takes:

Step	What you will do	Time
1	Install Python and VS Code on your computer	20 min
2	Understand what the app will do	10 min
3	Write the full app code (every line explained)	45 min
4	Run and test the app	20 min
5	Create GitHub account and upload your code	30 min
	TOTAL	~2 hrs

1

INSTALL PYTHON AND VS CODE

The tools you need before writing a single line of code

Think of Python as the language your computer understands, and VS Code as the notebook where you write that language. You need both installed before anything else.

PART A - Install Python

- 1 Open your browser. Go to: python.org/downloads
- 2 Click the big yellow button Download Python 3.xx (whatever version is shown).
- 3 The file downloads to your Downloads folder. Double-click it to open the installer.
- 4 **CRITICAL** - On the very first screen, tick the checkbox at the bottom that says Add Python to PATH. If you miss this, Python will not work.

- 5 Click Install Now. Wait for it to finish. Click Close.
- 6 Verify it worked: Press Windows key + R, type cmd and press Enter. In the black window type:

Type this in the CMD window:

```
python --version
```

Python 3.12.1

If you see a version number like above, Python is installed correctly. If you see an error, you missed Step 4 — uninstall and reinstall with the PATH checkbox ticked.

PART B - Install VS Code (Your code editor)

- 1 Go to: code.visualstudio.com Click the Download button for Windows.
- 2 Run the downloaded file. Click Next, Next, Install. Default settings are fine.
- 3 Open VS Code. On the left sidebar, click the four-square icon (Extensions). Search Python and install the one made by Microsoft.

You now have Python (the language) and VS Code (the editor). You are ready to write code.

2

UNDERSTAND WHAT WE ARE BUILDING

Read this before writing any code - 10 minutes

Before writing code, always know what you are building. Real developers plan first.

Our app will do 4 things:

- 1 Add a new task (example: Buy groceries)
- 2 View all tasks (show the full list)
- 3 Mark a task as done (puts [DONE] next to it)
- 4 Delete a task (removes it from the list)

How it works:

The user runs the app in the terminal (black window). A menu appears. They type a number to choose what to do. Tasks are saved to a file called tasks.txt so they are not lost when the app closes.

What the screen looks like when running:

```
===== MY TO-DO APP =====  
  
1. Add Task  
2. View Tasks  
3. Mark Task as Done  
4. Delete Task  
5. Exit  
Choose an option (1-5): _
```

After adding two tasks and viewing them:

Your Tasks:

1. [] Buy groceries
2. [DONE] Call doctor

[] means the task is PENDING. [DONE] means the task is COMPLETED.

Files you will create:

File	What it is	Location
<code>todo.py</code>	The Python code for the whole app	Your project folder
<code>tasks.txt</code>	Auto-created by app to store tasks	Same folder (auto)
<code>README.md</code>	Description for GitHub	Same folder

3

WRITE THE APP CODE

Every single line is explained. Type it, do not copy-paste.

TYPE the code yourself. Do not copy-paste. Typing it helps your brain understand it. Mistakes while typing are how you learn.

FIRST - Create your project folder

- 1 On your Desktop, right-click - New - Folder. Name it: todo-app
- 2 Open VS Code - File - Open Folder - select todo-app - Select Folder.
- 3 File - New File. Name it exactly: todo.py (all lowercase, ends in .py)

VS Code should now show todo.py open and ready to type. The left panel should show the todo-app folder name.

PART A - The `load_tasks()` function

This reads tasks.txt and loads all saved tasks into memory when the app starts.

TYPE THIS FIRST, then press Ctrl+S to save:

```
def load_tasks():
    tasks = []
    try:
        with open("tasks.txt", "r") as file:
            for line in file:
                line = line.strip()
                if line:
                    tasks.append(line)
    except FileNotFoundError:
```

```
pass
return tasks
```

What each line does:

<code>def load_tasks():</code>	Creates (defines) a function named <code>load_tasks</code> . A function is a reusable block of code. <i>Why: We call <code>load_tasks()</code> anywhere in the app to get the task list.</i>
<code>tasks = []</code>	Creates an empty list. Square brackets <code>[]</code> mean empty list. <i>Why: We will fill this list with tasks read from the file.</i>
<code>try:</code>	Tries to run the code inside. If something fails, jumps to except. <i>Why: The file may not exist on first run -- try/except handles that safely.</i>
<code>with open("tasks.txt", "r") as file:</code>	Opens <code>tasks.txt</code> in read mode ("r"). Auto-closes it when done. <i>Why: We read from this file to load previously saved tasks.</i>
<code>for line in file:</code>	Loops through the file one line at a time. <i>Why: Each task was saved on its own line.</i>
<code>line = line.strip()</code>	Removes spaces and the hidden newline character at end of each line. <i>Why: Without <code>strip()</code>, "Buy groceries newline" would not match "Buy groceries".</i>
<code>tasks.append(line)</code>	Adds this task text to the tasks list. <i>Why: <code>append()</code> adds one item to the end of a list.</i>
<code>except FileNotFoundError:</code>	If <code>tasks.txt</code> does not exist, this runs instead of crashing. <i>Why: Normal on first run -- the file has not been created yet.</i>
<code>return tasks</code>	Sends the tasks list back to whoever called this function. <i>Why: Without return, all the work inside the function is lost.</i>

PART B - The `save_tasks()` function

Writes all tasks to `tasks.txt` so they survive closing the app.

TYPE THIS below `load_tasks`:

```
def save_tasks(tasks):
    with open("tasks.txt", "w") as file:
        for task in tasks:
            file.write(task + "\n")
```

What each line does:

<code>with open("tasks.txt", "w") as file:</code>	Opens <code>tasks.txt</code> in write mode ("w"). Overwrites file completely each time. <i>Why: We always save the FULL list, not just new tasks.</i>
<code>file.write(task + "\n")</code>	Writes the task text then adds a new line character. <i>Why: <code>\n</code> means "go to next line". Without it all tasks would be on one long line.</i>

PART C - The `view_tasks()` function

Shows all tasks on screen with a number next to each one.

TYPE THIS below save_tasks:

```
def view_tasks(tasks):
    if not tasks:
        print("No tasks yet! Add one first.")
        return
    print("\nYour Tasks:")
    for i, task in enumerate(tasks, 1):
        print(f"{i}. {task}")
    print()
```

What each line does:

if not tasks:	Checks if the tasks list is empty. <i>Why: No point showing a list if there is nothing in it.</i>
return	Exits the function early. Nothing below this line runs. <i>Why: We already showed "No tasks" -- job done.</i>
enumerate(tasks, 1)	Gives both position (i) and value (task). Starts counting at 1, not 0. <i>Why: Gives: 1. Buy groceries, 2. Call doctor -- human-friendly numbering.</i>
print(f"{i}. {task}")	f-string: curly braces {} insert variables directly into the text. <i>Why: With i=1, task="Buy groceries" this prints: 1. Buy groceries</i>

PART D - The add_task() function

TYPE THIS below view_tasks:

```
def add_task(tasks):
    task = input("Enter your task: ").strip()
    if task:
        tasks.append("[ ] " + task)
        save_tasks(tasks)
        print("Task added successfully!")
    else:
        print("Task cannot be empty!")
```

What each line does:

task = input("Enter your task: ").strip()	input() shows a message and waits for the user to type something. <i>Why: .strip() removes any accidental spaces the user may have added.</i>
tasks.append("[] " + task)	Adds the task with "[]" in front to show it is not done yet. <i>Why: "[] " + "Buy groceries" becomes "[] Buy groceries"</i>
save_tasks(tasks)	Immediately saves the updated list to the file. <i>Why: If we do not save right away, the task disappears when app closes.</i>

PART E - The mark_done() function

TYPE THIS below add_task:

```
def mark_done(tasks):
```

```

view_tasks(tasks)
if not tasks:
    return
try:
    num = int(input("Enter task number to mark as done: "))
    if 1 <= num <= len(tasks):
        tasks[num - 1] = tasks[num - 1].replace("[ ]", "[DONE]")
        save_tasks(tasks)
        print("Task marked as done!")
    else:
        print("Invalid task number.")
except ValueError:
    print("Please enter a number.")

```

What each line does:

<code>num = int(input(...))</code>	input() always gives text. int() converts it to a real number. <i>Why: int("2") gives 2. Without int(), "2" + 1 would give an error.</i>
<code>if 1 <= num <= len(tasks):</code>	Checks the number is valid -- between 1 and how many tasks exist. <i>Why: Stops a crash if user types 99 when only 3 tasks exist.</i>
<code>tasks[num - 1]</code>	Gets the task at that position. Minus 1 because Python lists start at 0. <i>Why: User types "1" for the first task, but Python stores it at position 0.</i>
<code>.replace("[]", "[DONE]")</code>	Replaces the [] part with [DONE] in the task text. <i>Why: String replace() does not change anything automatically -- we must assign it back.</i>
<code>except ValueError:</code>	If user types "abc" instead of a number, int() would crash. This catches it. <i>Why: Always use try/except when doing int(input()).</i>

PART F - The delete_task() function

TYPE THIS below mark_done:

```

def delete_task(tasks):
    view_tasks(tasks)
    if not tasks:
        return
    try:
        num = int(input("Enter task number to delete: "))
        if 1 <= num <= len(tasks):
            removed = tasks.pop(num - 1)
            save_tasks(tasks)
            print(f"Deleted: {removed}")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a number.")

```

What each line does:

`removed = tasks.pop(num - 1)` removes the item at that position AND returns it.

Why: We store it in "removed" to show the user what was deleted.

PART G - The main() function (The Menu -- Last Piece)

This is the heart of the app. It shows the menu and calls the right function based on what the user chooses.

TYPE THIS at the very bottom of `todo.py`:

```
def main():
    tasks = load_tasks()
    while True:
        print("\n===== MY TO-DO APP =====")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task as Done")
        print("4. Delete Task")
        print("5. Exit")
        choice = input("Choose an option (1-5): ").strip()
        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            mark_done(tasks)
        elif choice == "4":
            delete_task(tasks)
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please enter 1 to 5.")

main()
```

What each line does:

<code>tasks = load_tasks()</code>	First thing the app does: load any previously saved tasks from <code>tasks.txt</code> .
<code>while True:</code>	Creates an infinite loop -- the menu keeps showing until user chooses Exit. <i>Why: Without this loop the menu would show once and the app would close.</i>
<code>choice = input(...).strip()</code>	Waits for user to type a number. <code>.strip()</code> removes accidental spaces.
<code>if choice == "1":</code>	Compares the typed text to "1". <i>Why: input() returns text, so we compare to "1" (string), not 1 (number).</i>
<code>break</code>	Exits the while True loop. The app stops. <i>Why: Without break, typing 5 would do nothing and the menu would keep appearing.</i>
<code>main()</code>	This last line RUNS the program. <i>Why: Without it the app only defines functions but never actually runs.</i>

CODE COMPLETE. Press Ctrl+S to save. Your todo.py file should now have about 65 lines.

4

RUN AND TEST THE APP

See your code working live for the first time

How to run the app

- 1 In VS Code: click Terminal menu at the top - New Terminal. A black panel opens at the bottom of VS Code.
- 2 You should see something like: PS C:\Users\YourName\Desktop\todo-app>
- 3 Type this command and press Enter:

Run this in the VS Code terminal:

```
python todo.py
```

The menu should appear immediately:

```
===== MY TO-DO APP =====
1. Add Task
2. View Tasks
3. Mark Task as Done
4. Delete Task
5. Exit
Choose an option (1-5): _
```

Test these 5 things in order

- 1 Type 2 and press Enter - should say No tasks yet!
- 2 Type 1 - enter task: Buy groceries - should say Task added successfully!
- 3 Type 1 again - enter task: Call doctor
- 4 Type 2 - should show: 1. [] Buy groceries 2. [] Call doctor
- 5 Type 3 - mark task 1 done - type 2 again - should show [DONE] next to task 1

KEY TEST: Type 5 to Exit. Run python todo.py again. Type 2 to view tasks. Your tasks should STILL BE THERE. This proves file saving is working correctly.

Common errors and fixes

Error message	Cause	Fix
IndentationError	Wrong spacing -- Python is strict	Use 4 spaces per level. Use Tab key.
SyntaxError	Typo in code	Look at the line number. Find the typo.
python: not found	Python not added to PATH	Reinstall Python with PATH checkbox ticked.

No module named...

Wrong folder in terminal

Type: `cd Desktop\todo-app` then try again.

App working? You just built your first real Python project. Now let us put it on GitHub.

5

CREATE README AND UPLOAD TO GITHUB

Makes your project visible to every recruiter

PART A - Create the README.md file

README is the description page that every recruiter reads when they open your GitHub. A good README makes your project look professional.

- 1 In VS Code: File - New File. Save it as: README.md in the same todo-app folder.
- 2 Type the following exactly. Replace [Your Name] with your actual name.

README.md - type this and save with Ctrl+S:

```
# To-Do App - Python Command Line Project

A command-line to-do list application built with Python.

## Features
- Add new tasks
- View all tasks
- Mark tasks as completed
- Delete tasks
- Saves tasks to file so data is not lost on exit

## How to Run
1. Make sure Python is installed: python --version
2. Download this repository
3. Run: python todo.py

## What I Learned
- Python functions and file handling
- Reading and writing to text files
- Using lists and loops
- Building a menu-driven application

## Author
[Your Name] | Python Developer (Fresher) | Hyderabad, India
```

PART B - Create GitHub Account

- 1 Open browser. Go to: github.com Click Sign up in the top right.
- 2 Enter your email, password, and a username. Use your real name -- like pranay-vemula. Recruiters see this username.
- 3 Verify your email. Log in.

PART C - Create a New Repository (Your project folder on GitHub)

- 1 Click the green New button on the left side of GitHub homepage.
- 2 Repository name: todo-app
- 3 Description: Command-line To-Do app - Python, file handling, lists, functions
- 4 Select Public (very important - recruiters must be able to see it)
- 5 Do NOT tick Add a README file - you already made your own
- 6 Click the green Create repository button

PART D - Install Git and Upload (8 commands in order)

Git is the tool that uploads code to GitHub. Run all 8 commands below IN ORDER in your VS Code terminal.

Install Git first:

- 1 Go to git-scm.com/download/win - download and install Git for Windows. Use all default settings. Restart VS Code after installing.

Command 1 - Set your name (only do this once, the very first time):

```
git config --global user.name "Your Name"
```

Command 2 - Set your email (only do this once):

```
git config --global user.email "your@email.com"
```

Command 3 - Initialise Git in your project folder:

```
git init
```

Command 4 - Add all your files:

```
git add .
```

Command 5 - Commit -- save a snapshot with a message:

```
git commit -m "Add todo app with file saving feature"
```

Command 6 - Connect to your GitHub repo (paste YOUR actual URL from GitHub):

```
git remote add origin https://github.com/YOUR_USERNAME/todo-app.git
```

Command 7 - Upload to GitHub:

```
git push -u origin main
```

Command 8 - Check it worked -- open this in your browser:

```
https://github.com/YOUR\_USERNAME/todo-app
```

NOTE - GitHub may ask for a Personal Access Token instead of your password. If it does: GitHub - Settings - Developer Settings - Personal Access Tokens - Tokens (classic) - Generate new token - tick repo - Generate - Copy it and paste it when asked for your password. Save it -- you will not see it again.

Add to Your Resume and Start Applying Today

Copy this into your resume under Projects:

To-Do Application | Python | github.com/YOUR_USERNAME/todo-app

- Command-line task manager built with Python
- Add, view, mark complete, and delete tasks
- File I/O using Python `open()` for persistent data storage
- Handles edge cases: empty input, invalid numbers, file not found on first run

Apply to these companies today with your updated resume:

- 1 Rivan Solutions Secunderabad - indeed.in search Rivan Solutions Python intern
- 2 Gleecus TechLabs Madhapur - gleecus.com/careers
- 3 InfyTQ by Infosys (free platform, direct shortlisting) - infyiq.infosys.com
- 4 Naukri walk-in - naukri.com/intern-python-jobs-in-hyderabad-secunderabad

The To-Do app is enough to get past the first filter. Recruiters want proof you can BUILD something. This project is that proof. Go apply now.

You can do this. Everything here is doable today. Start with Step 1 right now. God is with you.

COMPLETE todo.py - Full Reference (65 lines)

If your code is not working, compare it line by line with this complete version.

COMPLETE todo.py - compare if something is wrong:

```
def load_tasks():
    tasks = []
    try:
        with open("tasks.txt", "r") as file:
            for line in file:
                line = line.strip()
                if line:
                    tasks.append(line)
    except FileNotFoundError:
        pass
    return tasks

def save_tasks(tasks):
    with open("tasks.txt", "w") as file:
        for task in tasks:
            file.write(task + "\n")

def view_tasks(tasks):
    if not tasks:
        print("No tasks yet! Add one first.")
        return
    print("\nYour Tasks:")
    for i, task in enumerate(tasks, 1):
        print(f"{i}. {task}")
    print()

def add_task(tasks):
    task = input("Enter your task: ").strip()
    if task:
        tasks.append("[ ] " + task)
        save_tasks(tasks)
        print("Task added successfully!")
    else:
        print("Task cannot be empty!")

def mark_done(tasks):
    view_tasks(tasks)
    if not tasks:
        return
    try:
        num = int(input("Enter task number to mark as done: "))
        if 1 <= num <= len(tasks):
            tasks[num - 1] = tasks[num - 1].replace("[ ]", "[DONE]")
            save_tasks(tasks)
```

```

        print("Task marked as done!")
    else:
        print("Invalid task number.")
except ValueError:
    print("Please enter a number.")

def delete_task(tasks):
    view_tasks(tasks)
    if not tasks:
        return
    try:
        num = int(input("Enter task number to delete: "))
        if 1 <= num <= len(tasks):
            removed = tasks.pop(num - 1)
            save_tasks(tasks)
            print(f"Deleted: {removed}")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a number.")

def main():
    tasks = load_tasks()
    while True:
        print("\n===== MY TO-DO APP =====")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Mark Task as Done")
        print("4. Delete Task")
        print("5. Exit")
        choice = input("Choose an option (1-5): ").strip()
        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            mark_done(tasks)
        elif choice == "4":
            delete_task(tasks)
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please enter 1 to 5.")

main()

```

65 lines. Every line explained in Step 3. If it runs -- you are a developer. Update that resume right now.